



**DEPARTMENT OF INFORMATION TECHNOLOGY
LAB MANUAL**

CS23432 – SoftwareConstruction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE

Thandalam, Chennai-602015

Name: S. SURIYA PRAKASH

Register No: 241001279

Year / Branch / Section: 2ND / IT / E

Semester: IV

Academic Year: 2025-2026

INDEX

S.No	Date	Title
1.	21/01/26	Azure Devops Environment Setup.
2.	28/01/26	Azure Devops Project Setup and User Story Management.
3.	04/02/26	Setting Up Epics, Features, And User Stories for Project Planning.
4.	11/02/26	Sprint Planning.
5.	18/02/26	Poker Estimation.
6.	25/02/26	Designing Class and Sequence Diagrams for Project Architecture.
7.	11/03/26	Designing Architectural and ER Diagrams for Project Structure.
8.	18/03/26	Testing – Test Plans and Test Cases.
9.	25/03/26	Load Testing and Pipelines.
10.	08/04/26	GitHub: Project Structure & Naming Conventions.

EXP NO: 1	<i>AZURE DEVOPS ENVIRONMENT SETUP</i>
------------------	--

Aim:

To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

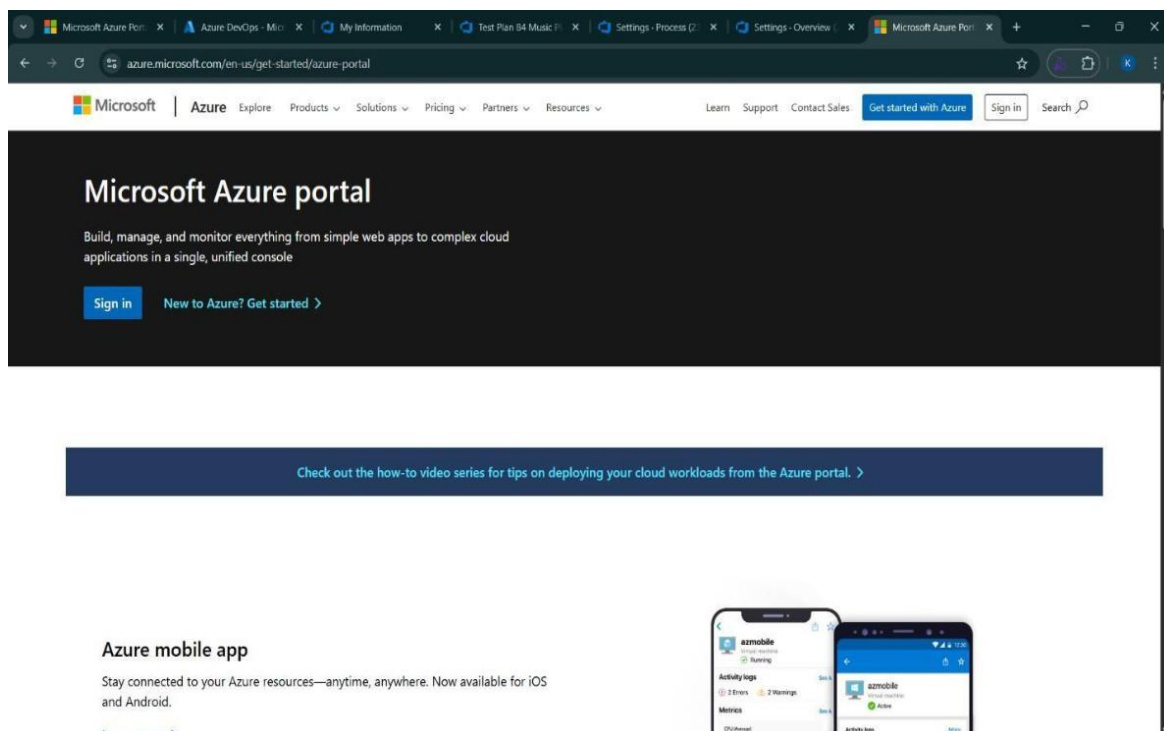
INSTALLATION:

1. Open your web browser and go to the Azure website:

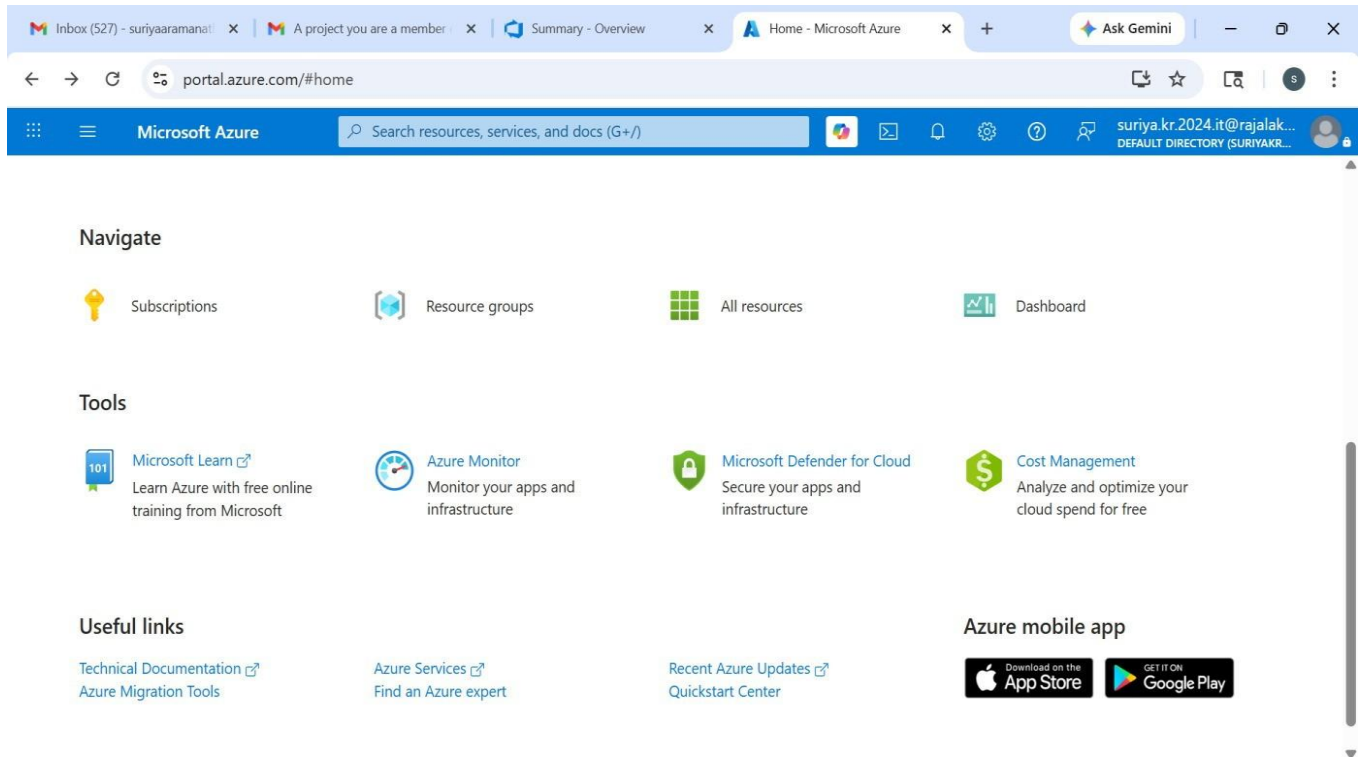
<https://azure.microsoft.com/en-us/get-started/azure-portal>.

Sign in using your Microsoft account credentials.

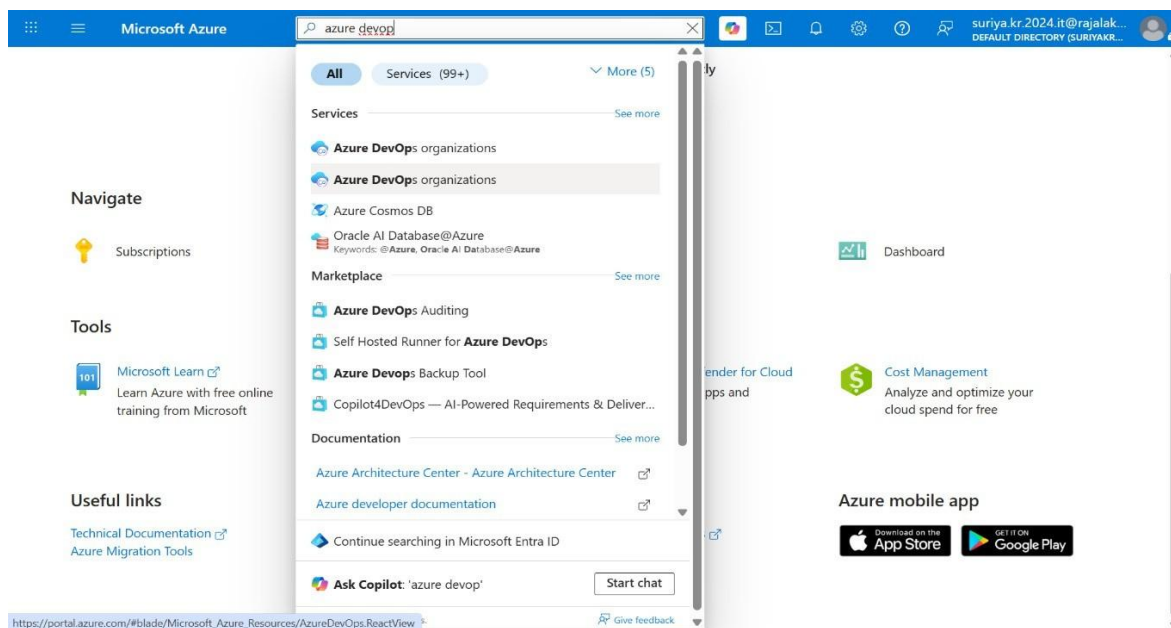
If you don't have a Microsoft account, you can create one here: <https://signup.live.com/?lic=1>



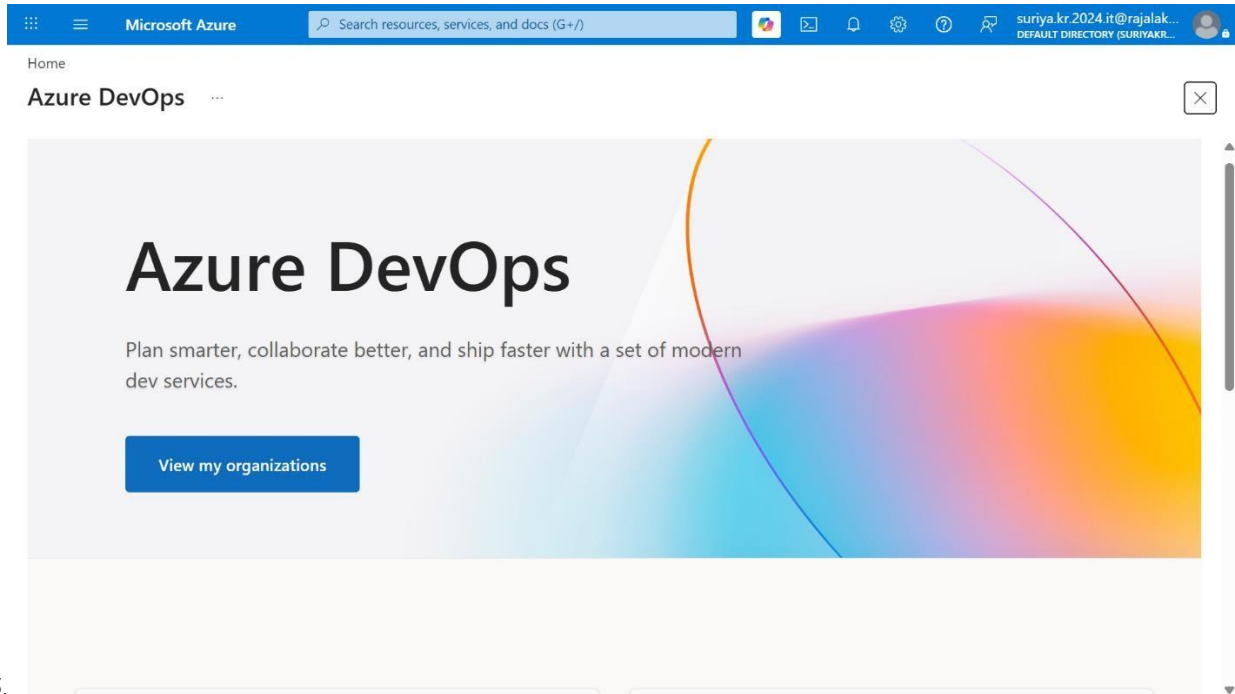
2. Azure home page



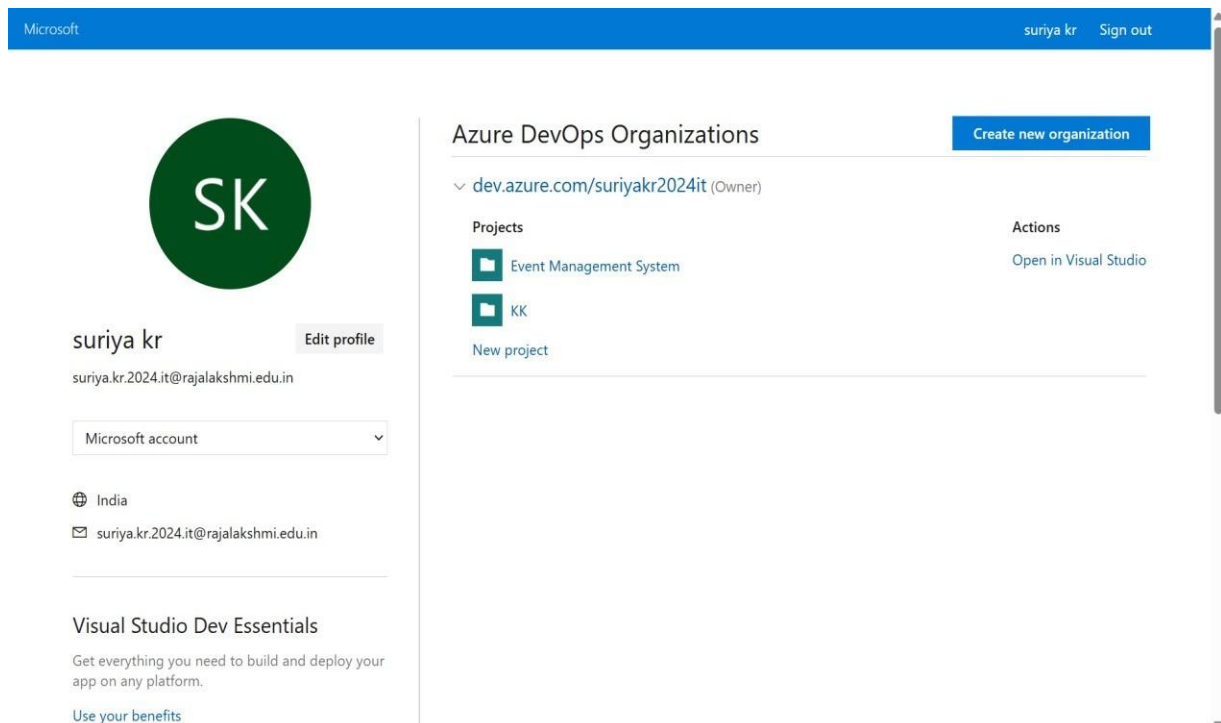
3. Open DevOps environment in the Azure platform by typing *Azure DevOps Organizations* in the search bar.



4. Click on the **View My Organization** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



5.



Result:

Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

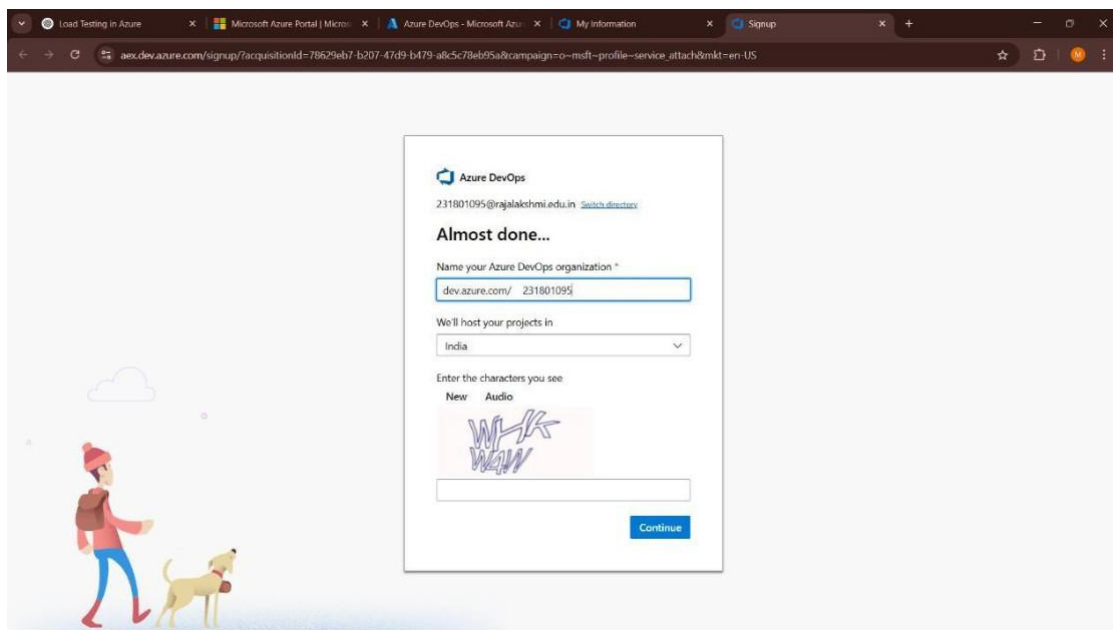
EXP NO: 2	<i>AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT</i>
------------------	--

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

Procedure:

1. Create An Azure Account



a. After the organization is set up, you'll need to create your first project. This is where you'll begin to manage code, pipelines, work items, and more.

b. On the organization's Home page, click on the ***New Project*** button.

c. Enter the project name, description, and visibility options:

Name: Event Management System Project

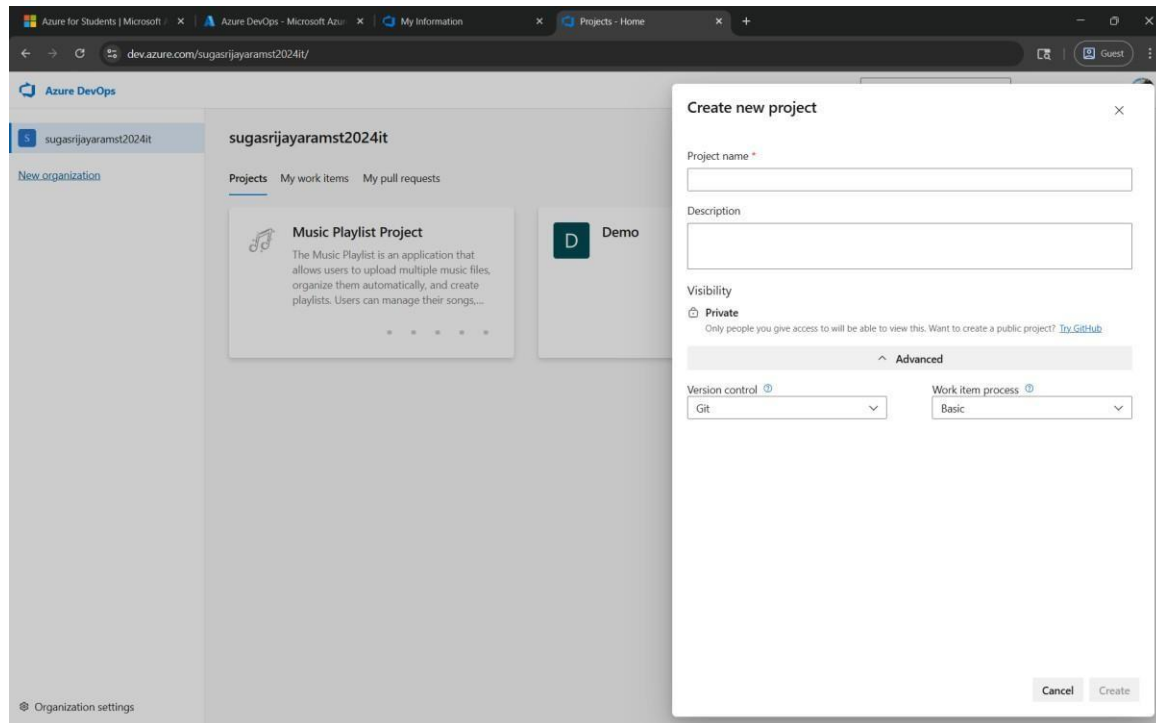
Description:

The Event Management System is an application that allows users to create, organize, and manage various events efficiently. Users can register for events, book tickets, manage schedules, and receive event notifications. The system also supports online payments, attendee management, and integration with external APIs for sending notifications and generating event analytics.

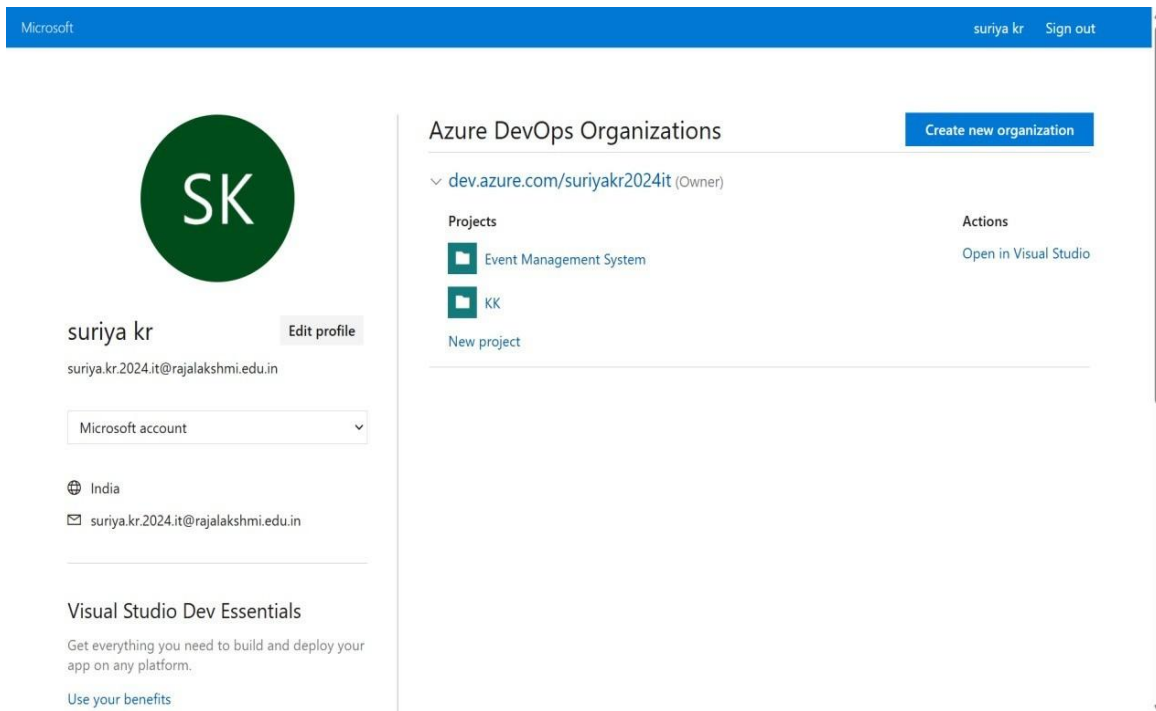
Visibility:

Choose whether you want the project to be ***Private*** (accessible only to those invited) or ***Public*** (accessible to anyone).

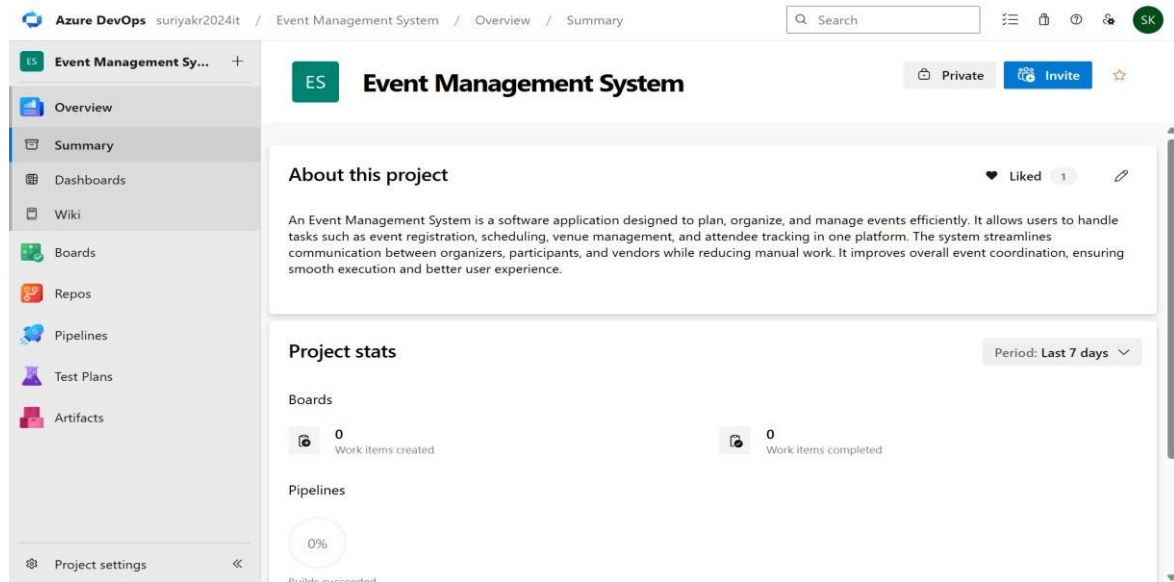
a. Once you've filled out the details, click **Create** to set up your first project.



2. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.

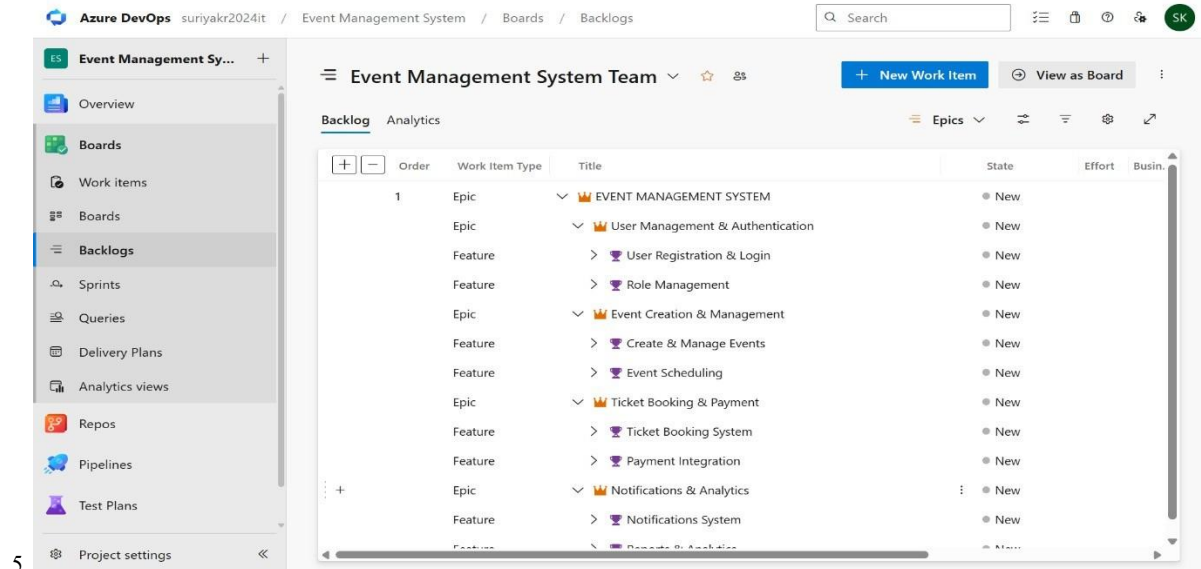


3. Project dashboard



4. To manage user stories:

- From the *left-hand navigation menu*, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.
- On the *work items* page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a + button or **Add New Work Item** depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.



5.

Browser tabs: Inbox (527) x, A project you x, Summary - Ov x, Azure DevOps x, My Informati x, Event Manage x, Ask Gemini

Address bar: dev.azure.com/suriyagr2024it/Event%20Management%20System/_backlogs/backlog/Event%20Management%20System%20Team/Epics?w...

EPIC 35

35 EVENT MANAGEMENT SYSTEM

No one selected | 0 Comments | Add Tag | Save and Close | Follow | Updated by suriya kr: Apr 8

State: New | Area: Event Management System | Reason: New | Iteration: Event Management System

Details | 4 | 0

Description

A system that helps users create, manage, and organize events, including registrations, scheduling, and tracking event activities efficiently.

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

[switch to Markdown editor](#)

Planning

Priority: 2

Risk:

Effort:

Business Value:

Time Criticality:

Start Date:

Select a date...

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status

Result:

Successfully created an Azure DevOps project with user story management and agile workflow setup.

EXP NO: 3

SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING

Aim:

To learn about how to create epics, user story, features, backlogs for Music Playlist Project.

Create Epic, Features, User Stories, Task:

The screenshot displays the Azure DevOps interface for a project named 'Event Management System'. The left sidebar shows the navigation menu with 'Backlogs' selected. The main area shows the 'Backlog' view for the 'Event Management System Team'. The backlog is organized into a table with columns for Order, Work Item Type, Title, State, Effort, and Business. The items are categorized into Epics and Features, all with a state of 'New'.

Order	Work Item Type	Title	State	Effort	Busin.
1	Epic	EVENT MANAGEMENT SYSTEM	New		
	Epic	User Management & Authentication	New		
	Feature	User Registration & Login	New		
	Feature	Role Management	New		
	Epic	Event Creation & Management	New		
	Feature	Create & Manage Events	New		
	Feature	Event Scheduling	New		
	Epic	Ticket Booking & Payment	New		
	Feature	Ticket Booking System	New		
	Feature	Payment Integration	New		
	Epic	Notifications & Analytics	New		
	Feature	Notifications System	New		
	Feature	Reports & Analytics	New		

1. Fill in Epics

dev.azure.com/suriyagr2024it/Event%20Management%20System/_backlogs/backlog/Event%20Management%20System%20Team/Epics?w...

Ask Gemini

47 Event Creation & Management

No one selected

0 Comments Add Tag

Save and Close

Follow

Updated by suriya kr: Apr 8

State	New	Area	Event Management System
Reason	New	Iteration	Event Management System

Details

3

0

Description

Allows organizers to create, update, and manage events.

Discussion

35

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

switch to Markdown editor

Planning

Priority

2

Risk

Effort

Business Value

Time Criticality

Start Date

Select a date...

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status

2. Fill in Features

Inbox (527) x A project you... x Summary - O... x Azure DevOps x My Informa... x Event Manag... x + Ask Gemini

dev.azure.com/suriyagr2024it/Event%20Management%20System/_backlogs/backlog/Event%20Management%20System%20Team/Epics?w... ☆ 🔍 ⓘ

EPIC 36

36 User Management & Authentication

No one selected Comments Add Tag Save and Close Follow ⚙️ ↺ ⌂ ⓘ

State	New	Area	Event Management System	
Reason	New	Iteration	Event Management System	Details ⌂ ⇄ 3 ✎ 0

Updated by suriya kr: Apr 8

Description

Handles user registration, login, roles, and secure authentication.

Discussion

5K

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

switch to Markdown editor

Planning

- Priority
2
- Risk
- Effort
- Business Value
- Time Criticality
- Start Date
Select a date...

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repo [commit](#), [pull request](#) or [branch](#) to see the status

3. Fill in User Story Details

EPIC 36*

36 User Management & Authentication

No one selected0 CommentsAdd Tag

Save and CloseFollow

StateNewAreaEvent Management SystemReasonNewIterationEvent Management System

Updated by suriya kr: Apr 8

Details30

Description

Handles user registration, login, roles, and secure authentication.
Enter the email id
Enter the correct password
Invalid shows error notification
Logout is functional is available
We support markdown, you can [convert this field](#).

Discussion

SK

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

switch to Markdown editor

Planning

Priority
2
Risk
Effort
Business Value
Time Criticality
Start Date
Select a date...
Target Date
Select a date...

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link
Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Classification

Result:

Thus, the creation of epics, features, user story and task has been created successfully.

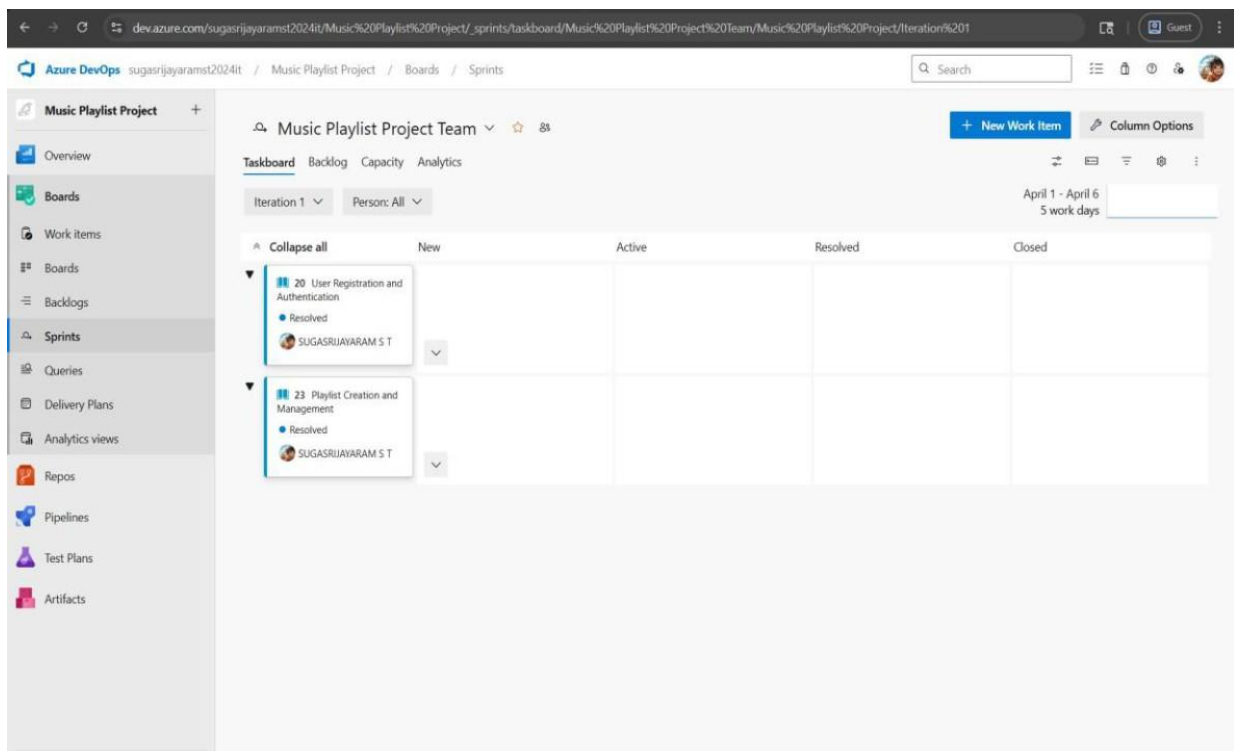
EXP NO: 4	<i>SPRINT PLANNING</i>
------------------	-------------------------------

Aim:

To assign user story to specific sprint for the Event Management System Project.

Sprint Planning:

Sprint 1



Sprint 2

Music Playlist Project

Overview | Boards | Work items | Boards | Backlogs | **Sprints** | Queries | Delivery Plans | Analytics views | Repos | Pipelines | Test Plans | Artifacts | Project settings

Music Playlist Project Team

Taskboard | Backlog | Capacity | Analytics

Iteration 2 | Person: All

April 7 - April 13
6 work days

	New	Active	Resolved	Closed
25 Fetch Song Metadata Using APIs Resolved sriram S				
22 Automatic Music Metadata Organization Resolved sriram S				

Sprint 3

Music Playlist Project

Overview | Boards | Work items | Boards | Backlogs | **Sprints** | Queries | Delivery Plans | Analytics views | Repos | Pipelines | Test Plans | Artifacts | Project settings

Music Playlist Project Team

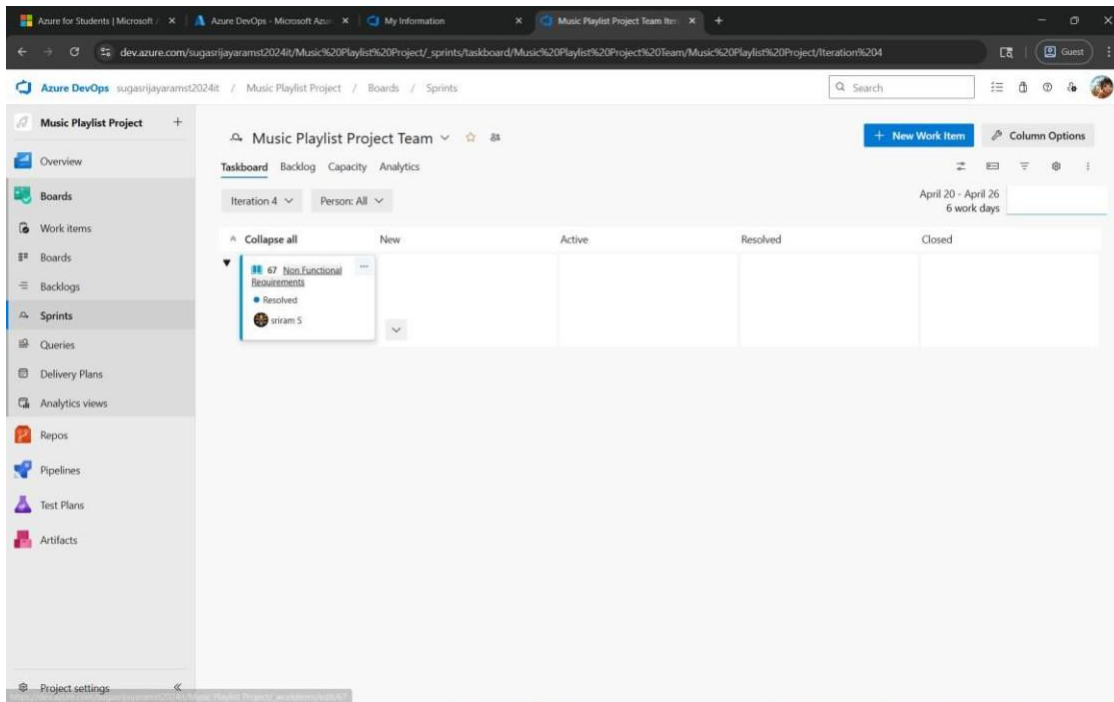
Taskboard | Backlog | Capacity | Analytics

Iteration 3 | Person: All

April 14 - April 20
6 work days

	New	Active	Resolved	Closed
24 Secure Song Download After Payment Resolved sriram S				
21 Upload Multiple Music Files Resolved sriram sathiyaseelan				

Sprint 4



Result:

The Sprints are created for the Event Management System Project.

EXP NO: 5	<i>POKER ESTIMATION</i>
------------------	--------------------------------

Aim:

Create Poker Estimation for the user stories for Event Management System Project.

Planning Poker Estimation:

User Story ID: 20

Title: User Registration and Authentication

Description:

As a user, I want to register and log in securely so that I can access my event and register for the required event.

Story Point Estimation: 8 Points

Team Poker Votes:

<i>Team</i>	<i>Member</i>	<i>Vote (Story Points)</i>
<i>Team</i>	<i>Member 1</i>	<i>5</i>
<i>Team</i>	<i>Member 2</i>	<i>8</i>
<i>Team</i>	<i>Member 3</i>	<i>8</i>
<i>Team</i>	<i>Member 4</i>	<i>8</i>

Final Consensus: 8 Points

Reason for Estimation:

Includes multiple features: registration, login, session, logout

Requires frontend + backend + database integration

Security implementation (password hashing, authentication)

Moderate complexity with high risk

Risks:

Security vulnerabilities

Session management issues

Integration challenges

Acceptance Criteria Covered:

- User registration with email & password
- Login with valid credentials
- Error message for invalid login
- Session maintained after login
- Logout functionality

Conclusion:

Final agreed estimation is **8 story points** based on complexity, effort, and risk.

The screenshot shows an Azure DevOps work item interface. At the top, the title is '36 User Management & Authentication'. Below the title, there are buttons for 'Save and Close', 'Follow', and 'Add Tag'. The work item is categorized under 'Event Management System'. The main content area is divided into three sections: 'Description', 'Planning', and 'Deployment'. The 'Description' section contains a list of acceptance criteria: 'Handles user registration, login, roles, and secure authentication.', 'Enter the email id', 'Enter the correct password', 'Invalid shows error notification', 'Logout is functional is available', and 'We support markdown, you can [convert this field](#).' The 'Planning' section includes fields for 'Priority' (2), 'Risk', 'Effort', 'Business Value', 'Time Criticality', 'Start Date', and 'Target Date'. The 'Deployment' section contains a text box with instructions on how to track releases and turn on deployment status reporting. Below the main content area, there is a 'Discussion' section with a comment box and a 'switch to Markdown editor' link. The bottom right corner shows a 'Related Work' section with an 'Add link' button.

EPIC 36*

36 User Management & Authentication

No one selected 0 Comments Add Tag

Save and Close Follow

State New Area Event Management System Updated by suriya kr: Apr 8

Reason New Iteration Event Management System Details 3 0

Description

Handles user registration, login, roles, and secure authentication.
Enter the email id
Enter the correct password
Invalid shows error notification
Logout is functional is available
We support markdown, you can [convert this field](#).

Planning

Priority
2
Risk
Effort
Business Value
Time Criticality
Start Date
Select a date...
Target Date
Select a date...

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link
Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.
[switch to Markdown editor](#)

Result:

The Estimation/Story Points is created for the project using Poker Estimation.

<i>EXP NO: 6</i>	<i>DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE</i>
-------------------------	--

Aim:

To Design a Class Diagram and Sequence Diagram for the Event Management System Project.

Class Diagram:

Classes:

User

Playlist

Song

Album

Payment

Wallet

Transaction

Feedback

Notification

Offer

Key Attributes:

User:

name, email, role, isPremium

Playlist:

name, songs[], isPublic

Song:

title, artist, duration, price

Album:

title, artist, songs[]

Wallet:

balance

Transaction:

type, amount

Relationships:

User → creates → Playlist

Playlist → contains → Song

Song → belongs to → Album

User → likes → Song

User → purchases → Song

User → owns → Wallet

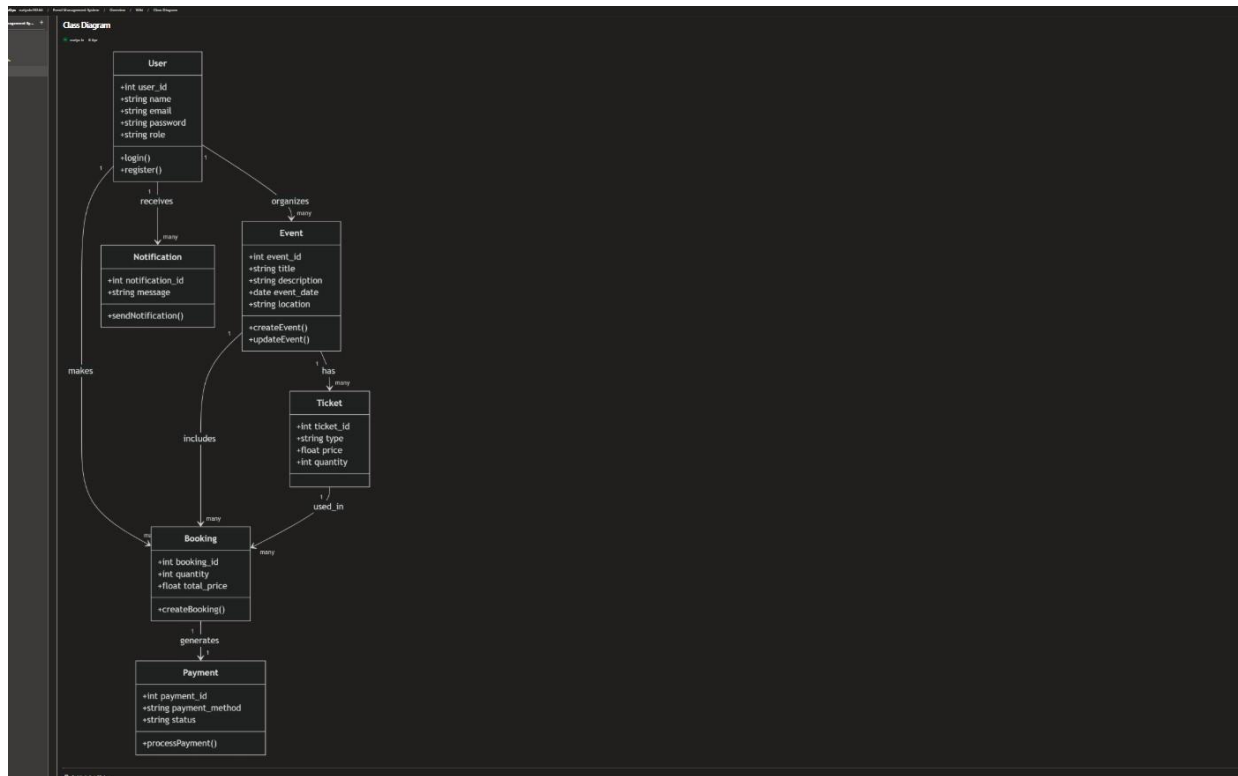
Wallet → records → Transaction

User → makes → Payment

User → submits → Feedback

User → receives → Notification

Offer → applies to → Song/Album



Sequence Diagram:

Actors:

User → Frontend → Backend → Database → Auth Service

Steps:

1. User enters email and password
2. Frontend validates input (email format, password)
3. Frontend sends login request to backend
4. Backend validates request data
5. Backend checks user in database

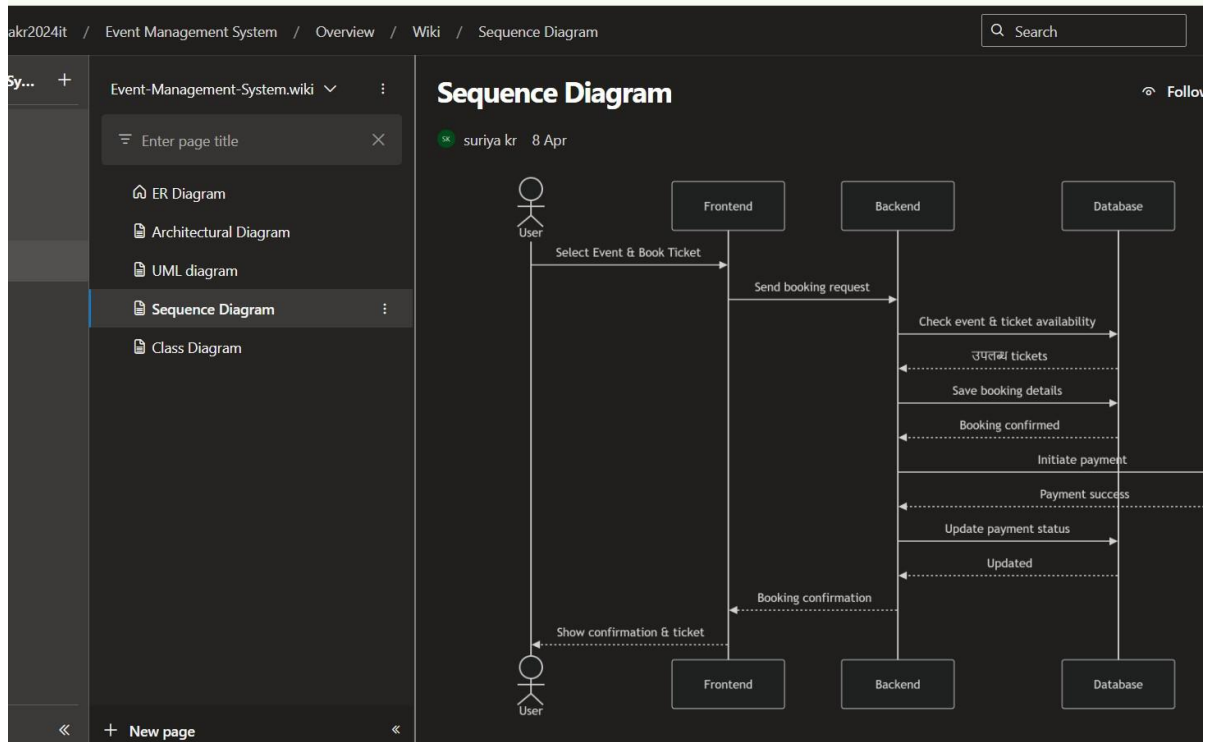
Condition: User Not Found

Backend returns error: *Invalid credentials*

Condition: User Found

6. Backend verifies password
7. If password incorrect → return error
8. If password correct → proceed
9. Backend verifies user with auth service
10. Backend generates JWT token
11. Backend sets session (token expiry)
12. Backend sends success response

13. Frontend stores token
14. Frontend redirects to dashboard
15. User login successful



Result:

The Class Diagram and Sequence Diagram is designed Successfully for the Event Management System Project.

EXP NO: 7	<i>DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE</i>
------------------	---

Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

Architectural Diagram:

Layers:

1. Client Layer

Browser

React App

Zustand (State Management)

2. API Communication

Axios (HTTP Requests)

3. Backend Layer

Express Server

Routes

Controllers

4. Services

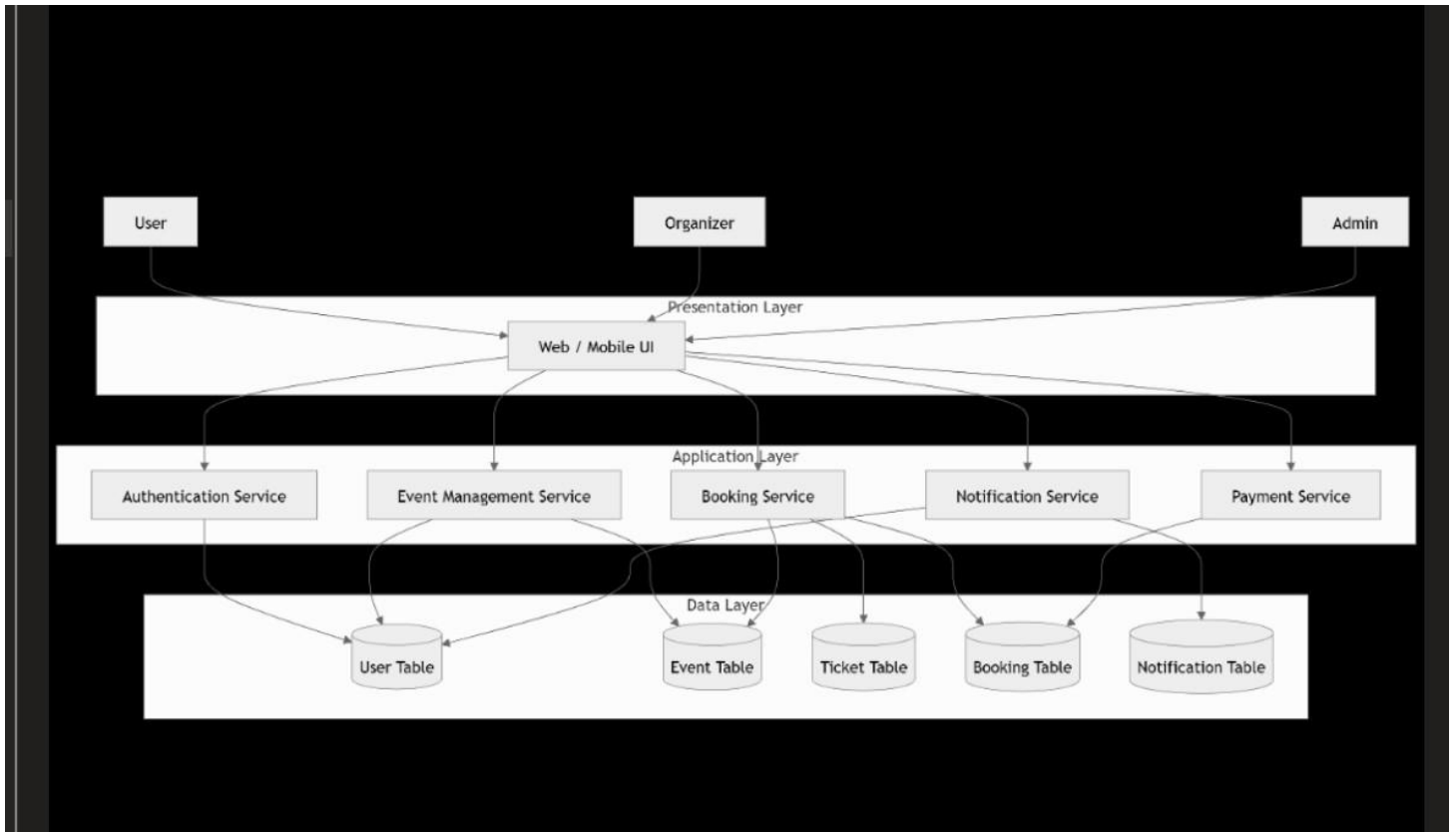
Clerk Authentication

File Upload (Cloudinary)

5. Database Layer

Mongoose Models

MongoDB



ER Diagram:

Entities & Attributes:

User

userId

fullName

email

clerkId

role

isPremium

subscriptionPlan

subscriptionExpiry

Playlist

playlistId
name
description
isPublic
creatorId

Song

songId
title
artist
genre
duration
price
albumId

Album

albumId
title
artist
genre
releaseYear

Wallet

☐ userId

☐ balance

Transaction

transactionId

amount

type

Payment

paymentId

userId

amount

status

Offer

offerId

name

discountPercentage

startDate

endDate

Feedback

feedbackId

userId

message

rating

Notification

notificationId

userId

message

type

Announcement

announcementId

title

content

targetRoles

expiresAt

Relationships

User *creates* Playlist (1 : M)

Playlist *contains* Song (M : N)

Song *belongs to* Album (M : 1)

User *likes* Song (M : N)

User *purchases* Song (M : N)

User *owns* Wallet (1 : 1)

Wallet *records* Transaction (1 : M)

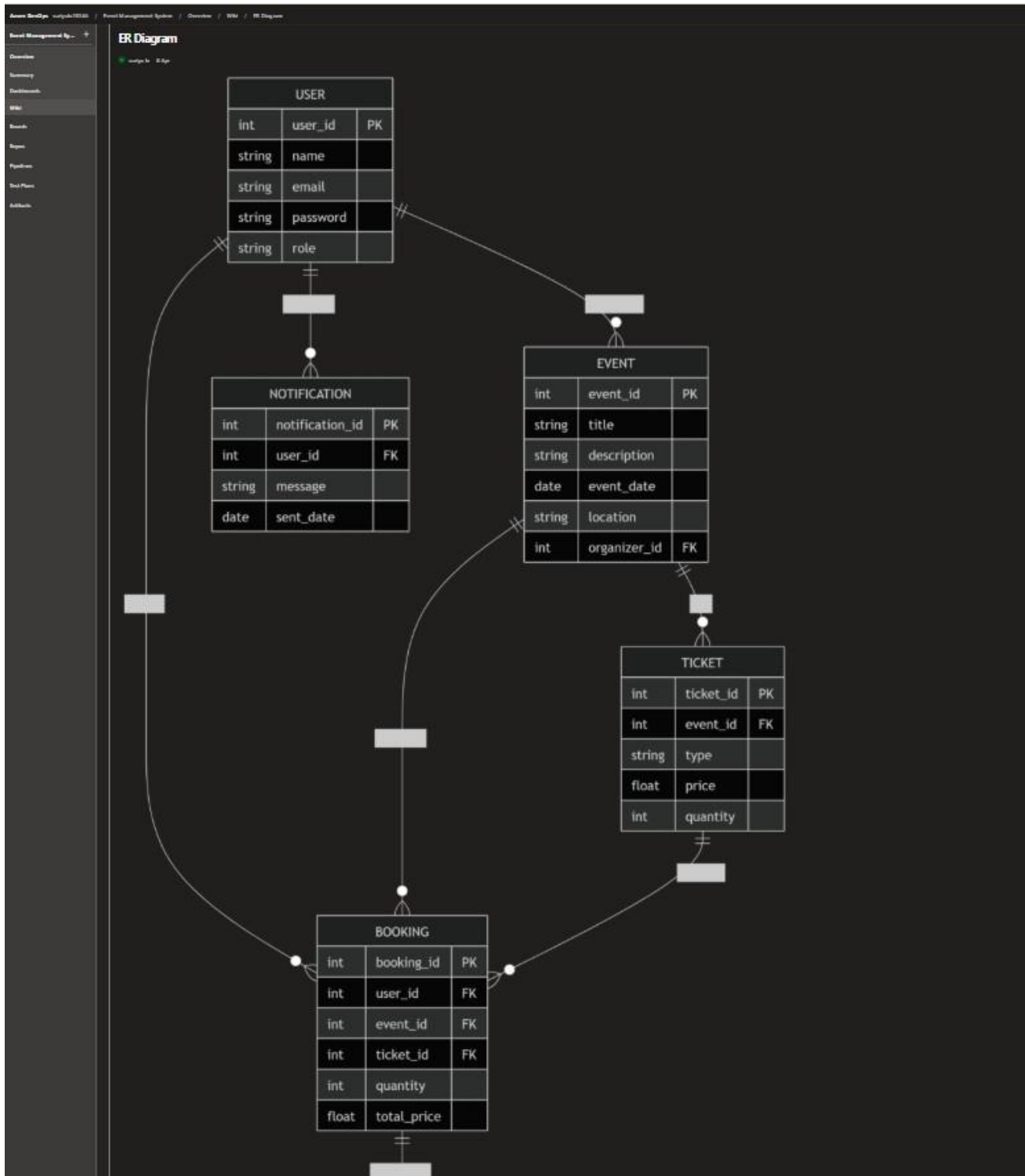
User *makes* Payment (1 : M)

Offer *applies to* Song / Album

User *submits* Feedback

User *receives* Notification

Announcement *targets* User



Result:

The Architecture Diagram and ER Diagram is designed Successfully for the Event Management System Project.

EXP NO: 8	TESTING – TEST PLANS AND TEST CASES
------------------	--

Aim:

Test Plans and Test Case and write two test cases for at least two user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case:

Test Case Design Procedure

1. Understand Core Features of the Application

- User Signup & Login
- Viewing and Managing Playlists
- Fetching Real-time Metadata
- Editing playlists (rename, reorder, record)
- Creating smart audio playlists based on categories (mood, genre, artist, etc.)

2. Define User Interactions

- Each test case simulates a real user behaviour (e.g., logging in, rebooking the event register).

3. Design Happy Path Test Cases

- Focused on validating that all features function as expected under normal conditions.
- Example: User logs in successfully, adds item to playlist, or creates a category-based playlist.

4. Design Error Path Test Cases

- Simulate negative or unexpected scenarios to test robustness and error handling.
- Example: Login fails with invalid credentials, save fails when offline, no recommendations found.

5. Break Down Steps and Expected Results

- Each test case contains step-by-step actions and a corresponding expected outcome.
- Ensures clarity for both testers and automation scripts.

6. Use Clear Naming and IDs

- Test cases are named clearly (e.g., TC01 – Successful signup, TC06 – Add and Remove event register).
- Helps in quick identification and linking to user stories or features.

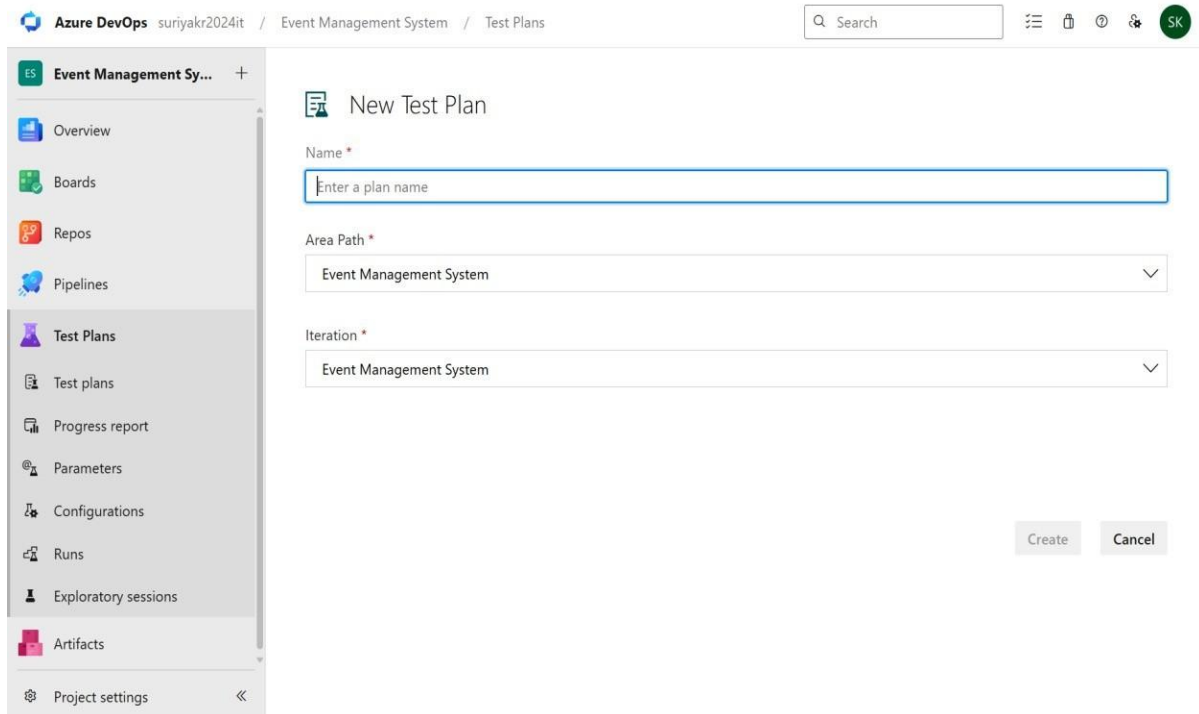
7. *Separate Test Suites*

- Grouped test cases based on functionality (e.g., Login, event booking Editing, Recommendation System).
- Improves organization and test execution flow in Azure DevOps.

8. *Prioritize and Review*

- Critical user actions are marked high-priority.
- Reviewed for completeness and traceability against feature requirements.

1. *New test plan*



The screenshot shows the 'New Test Plan' interface in Azure DevOps. The top navigation bar includes the Azure DevOps logo, the user 'suriykr2024it', the project path 'Event Management System / Test Plans', a search bar, and utility icons. The left sidebar contains a menu with options: Overview, Boards, Repos, Pipelines, Test Plans (selected), Test plans, Progress report, Parameters, Configurations, Runs, Exploratory sessions, Artifacts, and Project settings. The main content area is titled 'New Test Plan' and contains three required fields: 'Name' (with a placeholder 'Enter a plan name'), 'Area Path' (set to 'Event Management System'), and 'Iteration' (set to 'Event Management System'). At the bottom right are 'Create' and 'Cancel' buttons.

Test case

Give two test cases for at least two user stories showcasing the happy path and error scenarios in azure DevOps platform.

Event Management Project – Test Plans

USER STORIES

- As a user, I want to register and log in securely so that I can access my event register and do booking. (ID: 20).
- As a user, I want to book and save so that I can organize my event easily (ID: 23).

Test Suites

Test Suit: TS01 - User Registration and Authentication (ID: 20)

1. TC01 – User Registration Successfully w/o Error

Action:

- o Go to the Sign-Up page
- o Enter valid name, email, and password
- o Click "Sign Up"

Expected Results:

- o Sign-Up form is displayed
- o Fields accept values without error
- o Account is created successfully
- o User is redirected to dashboard

Type: Happy Path

2. TC02 – User Registration Failed with Invalid User Details

Action:

- o Go to the Sign-Up page
- o Enter invalid details (empty fields / wrong email format / weak password)
- o Click "Sign Up"

Expected Results:

- o Validation errors are displayed
- o Account is not created

Type: Error Path

3. TC03 – User Login Successfully with Valid Credentials

Action:

- o Go to Login page
- o Enter valid email and password
- o Click "Login"

Expected Results:

- o Login form is displayed
- o Credentials are accepted
- o User is redirected to dashboard

Type: Happy Path

4. TC04 – Login with Invalid Credentials

Action:

- o Go to Login page
- o Enter invalid email or password
- o Click "Login"

Expected Results:

- o Input is accepted
- o Error message "Invalid username or password" is shown

Type: Error Path

Test Suit: TS02 – event booking and Management (ID: 23)

1. TC05 – event booking Successfully

Action:

- o Login with valid credentials
- o Navigate to "My Playlists"
- o Click "Create Playlist"
- o Enter playlist name
- o Click "Save/Create"

Expected Results:

- o Playlist is created successfully
- o Playlist appears in list

Type: Happy Path

2. TC06 – Add and Remove event from save

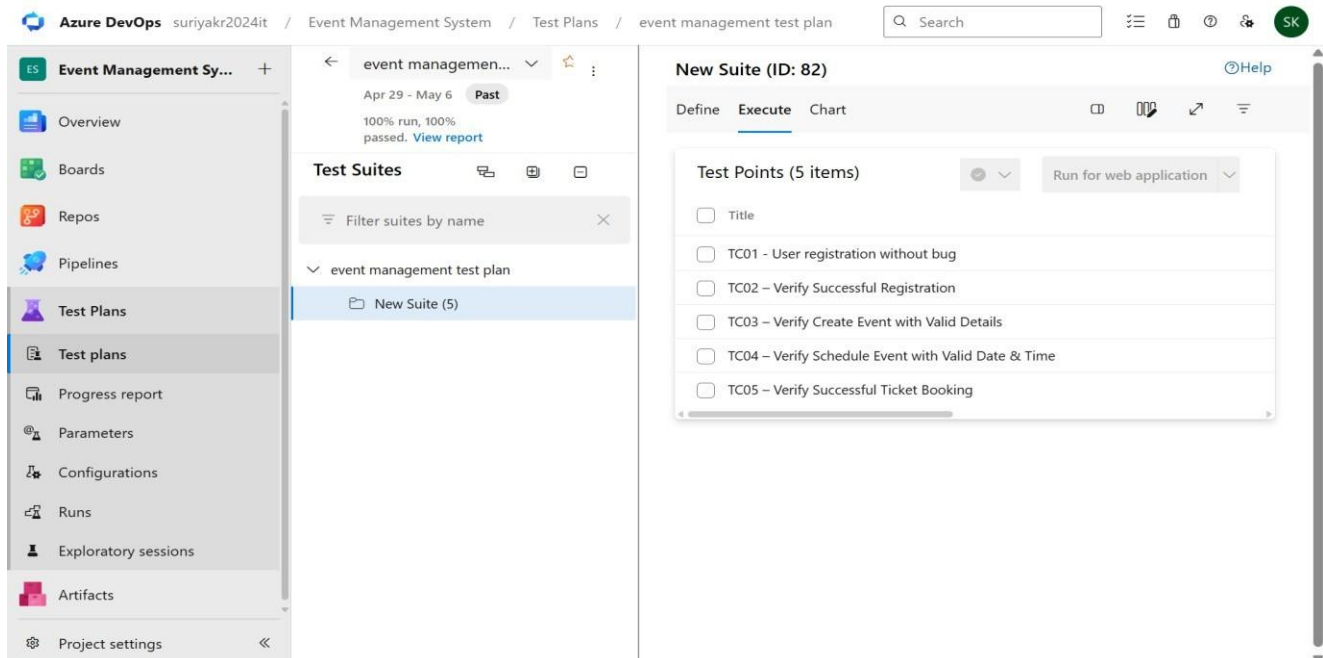
- o **Action:**
 - o Open an existing folder
 - o Click "Add event" and select a event
 - o Click "Remove" on a event
- o **Expected Results:**
 - o Song is added to save
 - o Song is removed from save
- o **Type:** Happy Path

3. TC08 – Delete booking and Save Changes

- o **Action:**
 - o Navigate to "My events"
 - o Click "Delete" on a events
 - o Confirm deletion
- o **Expected Results:**
 - o event is deleted successfully
 - o event is removed from list
 - o Changes are saved

Type: Happy Path

Test Cases



ES

Event Management Sy...

+

Overview

Boards

Repos

Pipelines

Test Plans

Test plans

Progress report

Parameters

Configurations

Runs

Exploratory sessions

Artifacts

Project settings

←

event managemen...

▼

⌵

⌵

Apr 29 - May 6

Past

100% run, 100% passed.

View report

Test Suites

Filter suites by name

▼ event management test plan

New Suite (5)

Test point ID 1 execution history

↑

↓

×

Test Case

83 TC01 - User registration without bug

Test Suite

82 New Suite

Current Tester

SK

suriya kr

Configuration

Windows 10

Outcome	Test run	Run by	Time completed
✔ Passed	16	suriya kr	Apr 29
✔ Passed	3	suriya kr	Apr 29
✖ Failed	2	suriya kr	Apr 29

Currently showing a test point execution history. View all history for a full test case execution history.

View all history

TEST CASE 85

85 TC06 – Add and Remove Songs from Playlist

SUGASRIJAYARAM S T

0 Comments Add Tag

State Closed

Area Music Playlist Project

Reason Obsolete

Iteration Music Playlist Project\Iteration 1

Save and Close

Follow

Updated by SUGASRIJAYARAM S T: 43m ago

Steps

Summary

Associated Automation

2

0

Steps

Recent test results

Deployment

Development

Add link

Related Work

1. Open an existing playlist

2. Click "Add Song"

3. Select a song

4. Verify playlist

5. Click "Remove/Delete" on a song

6. Verify playlist

Expected result

Attachments

Playlist details are displayed

Song selection options appear

Song is added to the playlist

Added song is visible in the list

Song is removed

Removed song no longer appears

Click or type here to add a step

Parameter values

Passed by with Windows 10

Completed 4:36 PM, 4/29/2026

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Add link

Parent

23 Playlist Creation and Management

Updated 54m ago Resolved

Tests

23 Playlist Creation and Management

4. Running the test cases

The screenshot shows the Azure DevOps Test Plans interface. The left sidebar contains navigation options: Overview, Boards, Repos, Pipelines, Test Plans (selected), Progress report, Parameters, Configurations, Runs, Exploratory sessions, and Artifacts. The main area displays the 'Music Playlist Test Plan 1' with a 'Past' status and '100% run, 100% passed' result. A 'Test Suites' filter is applied. The 'Test Points (3 items)' table lists three test cases, all of which are 'Passed'.

Title	Outcome	Order	Test Case Id	Status
TC05 - Create Playlist Successfully	Passed	1	84	Completed
TC06 - Add and Remove Songs from Playlist	Passed	2	85	Completed
TC08 - Delete Playlist and Save Changes	Passed	3	86	Completed

A context menu is open for the selected test case 'TC06 - Add and Remove Songs from Playlist', showing options: View execution history, Mark Outcome, Run, Reset test to active, Edit test case, Assign tester, and View test result. The 'Run' option is expanded, showing sub-options: Run for web application, Run for desktop application, and Run with options.

The screenshot shows the Azure DevOps Test Runner interface for the test case '85: TC06 - Add and Remove Songs from Playlist'. The test steps are listed with their expected results and actual outcomes, all of which are 'Passed'.

Step	Expected Result	Actual Result
1. Open an existing playlist	Playlist details are displayed	Passed
2. Click "Add Song"	Song selection options appear	Passed
3. Select a song	Song is added to the playlist	Passed
4. Verify playlist	Added song is visible in the list	Passed
5. Click "Remove/Delete" on a song	Song is removed	Passed
6. Verify playlist	Removed song no longer appears	Passed

5. Creating the bug

The screenshot shows the Azure DevOps Test Plans interface. On the left, a test case titled "85*: TC06 - Add and Remove Songs from Playlist" is displayed. It contains six steps, each with an expected result and a comment field. The test case is currently in a "Failed" state. On the right, a table shows the test results for the test case. The table has columns for Outcome, Order, Test Case Id, and Status. The results are as follows:

Outcome	Order	Test Case Id	Status
Passed	1	84	Cl
Passed	2	85	Cl
Passed	3	86	Cl

The screenshot shows the Azure DevOps Bug report interface for Bug01- TC06 Add and Remove Songs from playlist. The bug is currently in a "New" state. The interface includes a "Repro Steps" section, a "Planning" section, a "Deployment" section, a "Development" section, a "Related Work" section, a "System Info" section, and a "Discussion" section.

Repro Steps

Step no.	Result	Title
1.	Passed	Open an existing playlist
2.	Failed	Click "Add Song"
3.	Failed	Select a song

Planning

Resolved Reason

Story Points

Priority

2

Severity

3 - Medium

Activity

Effort (Hours)

Original Estimate

Remaining

Completed

Planning Poker

Save and re-open the new work item to start estimation.

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also create a branch to get started.

Related Work

Add link

Parent

Playlist Creation and Management

Updated 39m ago

Resolved

Tested By

85 TC06 - Add and Remove Songs from Playlist

Updated 49m ago

Closed

System Info

Found in Build

Integrated in Build

dev.azure.com/suriykr2024it/Event%20Management%20System/_backlogs/backlog/Event%20Management%20System%20Team/Epics?w...

EPIC 47

47 Event Creation & Management

No one selected 0 Comments Add Tag

Save and Close Follow

Updated by suriya kr: Apr 8

State: New Area: Event Management System Reason: New Iteration: Event Management System

Description

Allows organizers to create, update, and manage events.

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

[switch to Markdown editor](#)

Planning

Priority: 2 Risk: Effort: Business Value: Time Criticality: Start Date: Select a date...

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status

6. Bug report

- Assigning bug to the developer and changing state to Active

Azure DevOps suriykr2024it / Event Management System / Test Plans

Search

Event Management Sy... +

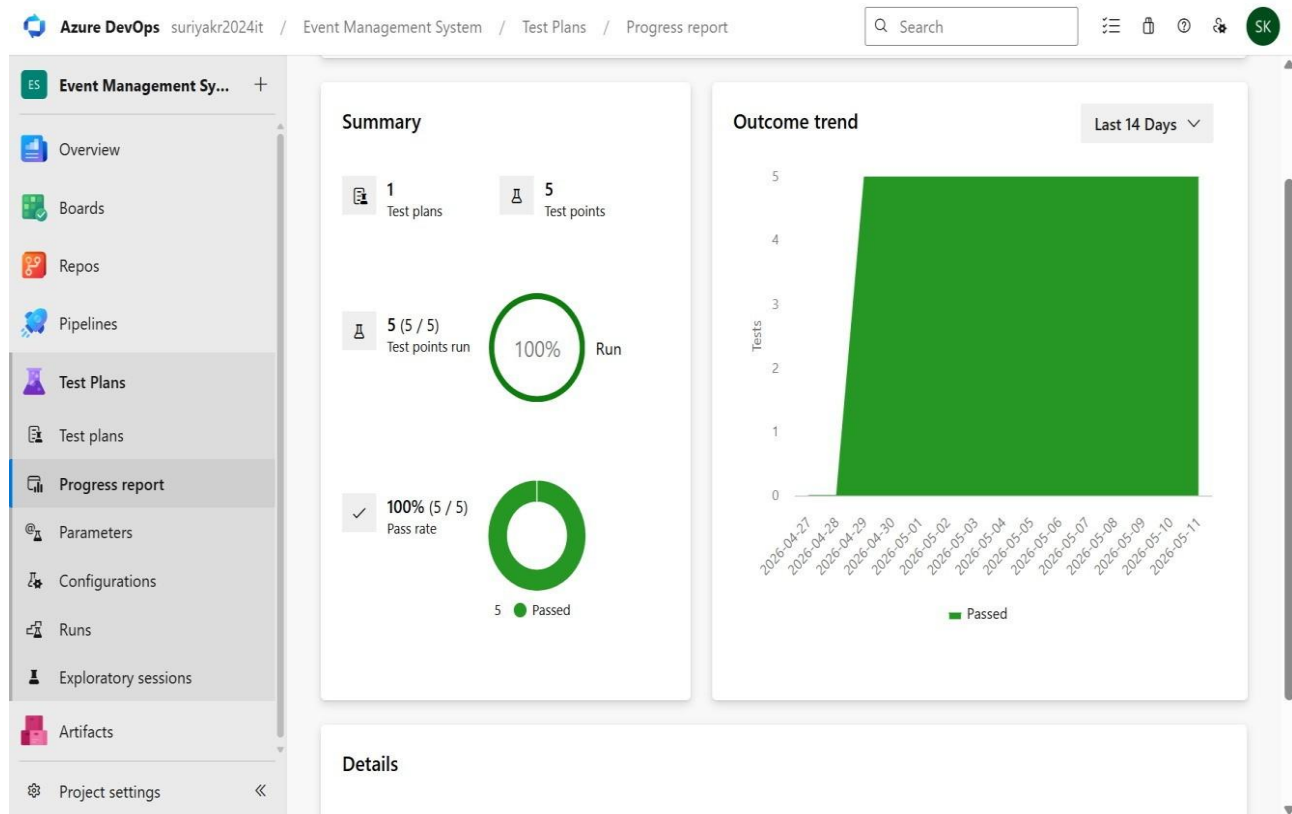
Overview Boards Repos Pipelines **Test Plans** Test plans Progress report Parameters Configurations Runs Exploratory sessions Artifacts Project settings <<

83 TC01 - User registration without bug

View test case

Outcome Windows 10 suriya kr event management test plan New Suite < > X

Outcome	Test run ID	TimeStamp	Configuration	Run by	Current tester	Test plan	Test suite	Iter...
Passed	16	Apr 29	Windows 10	suriya kr	suriya kr	event man...	82 Ne...	Eve
Passed	3	Apr 29	Windows 10	suriya kr	suriya kr	event man...	82 Ne...	Eve
Failed	2	Apr 29	Windows 10	suriya kr	suriya kr	event man...	82 Ne...	Eve



Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path.

EXP NO: 9	<i>LOAD TESTING AND PIPELINES</i>
------------------	--

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing

Azure Load Testing:

Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

1. Sign in to Azure Portal
Go to <https://portal.azure.com> and log in.
2. Create the Resource
 - o Go to **Create a resource** → Search for “Azure Load Testing”.
 - o Select Azure Load Testing and click Create.
3. Fill in the Configuration Details
 - o **Subscription:** Choose your Azure subscription.
 - o **Resource Group:** Create new or select an existing one.
 - o **Name:** Provide a unique name (no special characters).
 - o **Location:** Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

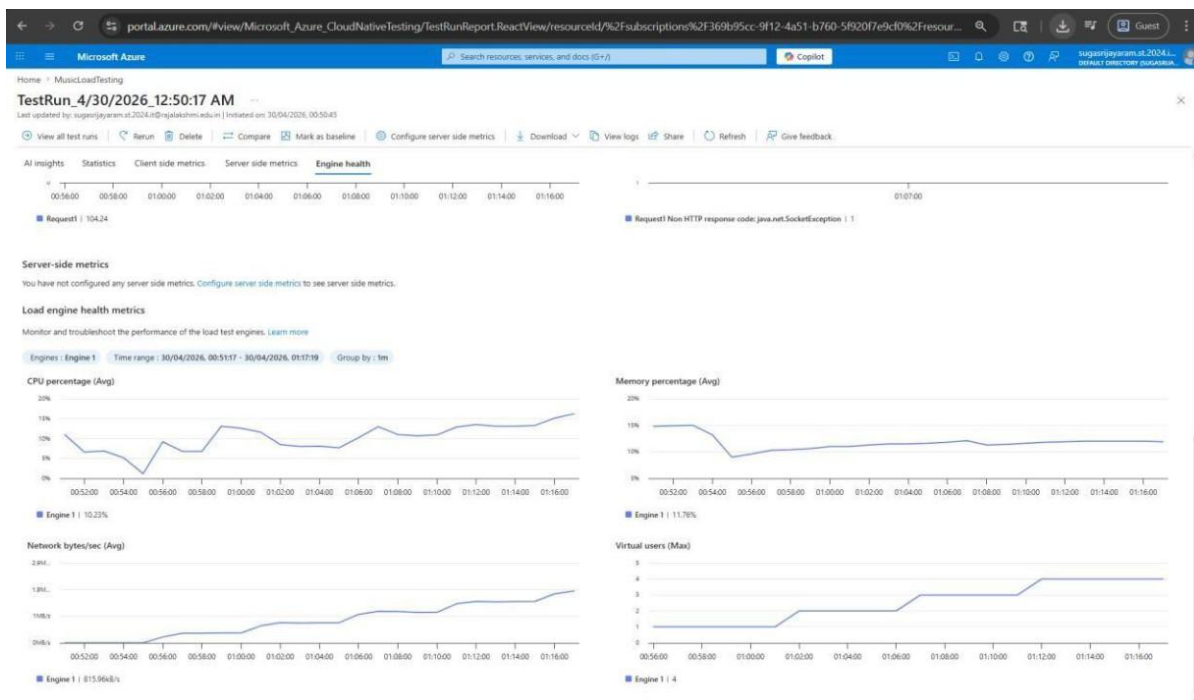
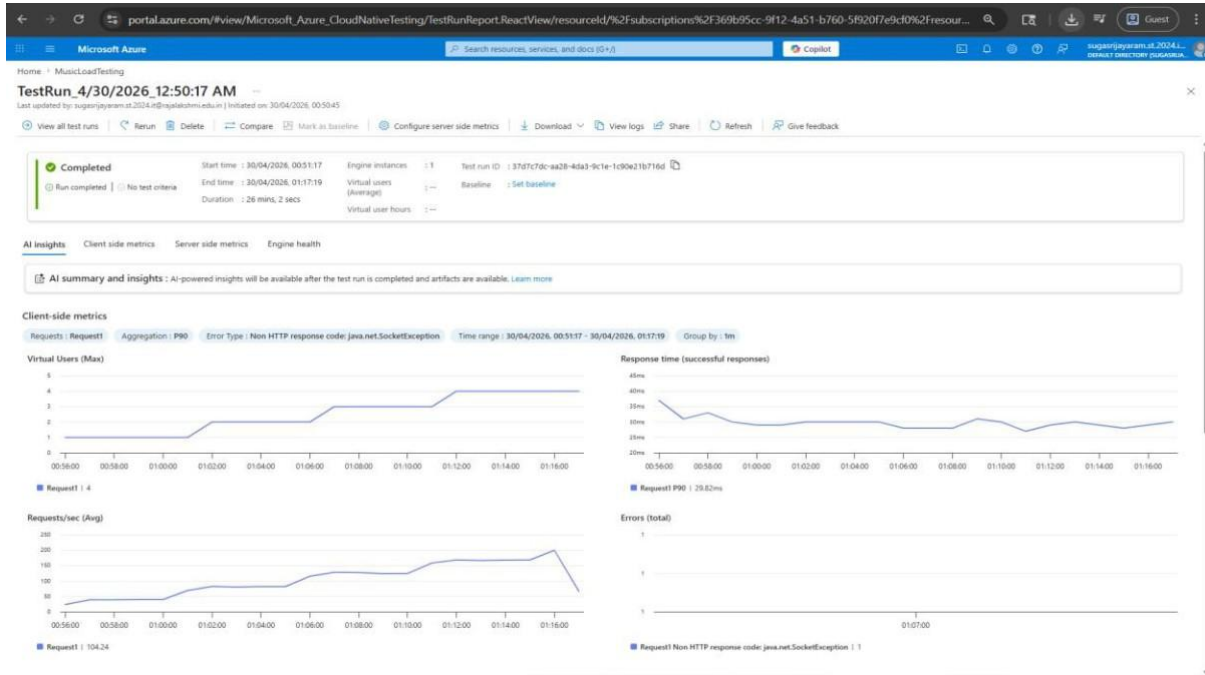
Steps to Create and Run a Load Test:

Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab
 - o **Test Name:** Provide a unique name.
 - o **Description:** (Optional) Add test purpose.
 - o **Run After Creation:** Keep checked.
3. Load Settings
 - o **Test URL:** Enter the target endpoint (e.g., <https://yourapi.com/products>).

4. Click Review + Create → Create to start the test.

Load Testing



Featured Events

Check out what's coming up next.

[View All →](#)



₹100

OTHER

kung fu panda movie

thriller

9/4/2026 at 18:00

chennai



₹1000

CONCERT

rec cultural

rahman musicals

16/4/2026 at 21:00

REC



₹150

CONFERENCE

Tech Conference 2026

The biggest tech conference of the year.

15/6/2026 at 09:00 AM

San Francisco, CA

nts/69d71b807ce4fde84227d85d

Pipelines

Description:

This experiment demonstrates how to connect a GitHub-hosted Flask-based music recommendation project with Azure DevOps. The pipeline will automatically install dependencies, run basic tests, and publish artifacts. This ensures that every commit triggers checks for reliability and smooth deployment.

Steps:

1. Connect GitHub to Azure DevOps:
 - In Azure DevOps, create a new project.
 - Create a pipeline and select GitHub as the source.
 - Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.
2. Create azure-pipelines.yml in Your Repo Root:
 - In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
 - Add the following basic pipeline configuration for Python and Flask:

yaml Code

trigger:

- main

pool:

vmImage: ubuntu-latest

stages:

```
# -----
# STAGE 1: Build Frontend (React + Vite)
# -----
- stage: BuildFrontend
  displayName: 'Build Frontend'
  jobs:
    - job: FrontendBuild
      displayName: 'Install & Build React App'
      steps:
        - task: NodeTool@0
          inputs:
            versionSpec: '20.x'
            displayName: 'Use Node.js 20'

        - script: |
            node --version
            npm --version
            cd frontend
            npm install
            npm run build
            displayName: 'npm install and build (frontend)'

        - task: PublishBuildArtifacts@1
          inputs:
            PathtoPublish: 'frontend/dist'
            ArtifactName: 'frontend-dist'
            publishLocation: 'Container'
            displayName: 'Publish Frontend Artifacts'

# -----
# STAGE 2: Build Backend (Node/Express)
# -----
- stage: BuildBackend
  displayName: 'Build Backend'
  dependsOn: BuildFrontend
  jobs:
    - job: BackendBuild
      displayName: 'Install Backend Dependencies'
      steps:
        - task: NodeTool@0
          inputs:
            versionSpec: '20.x'
            displayName: 'Use Node.js 20'
```

```
- script: |  
node --version  
cd server  
npm install --omit=dev  
displayName: 'npm install (server)'  
- task: PublishBuildArtifacts@1  
inputs:  
PathtoPublish: 'server'  
ArtifactName: 'backend'  
publishLocation: 'Container'  
displayName: 'Publish Backend Artifacts'
```

3. Run and Monitor Pipeline:

- o Commit changes to the main branch of your repository to trigger the pipeline in Azure DevOps.
- o Monitor the logs in the Azure DevOps portal to view logs, errors, or success messages and ensure everything runs smoothly.

Pipeline

dev.azure.com/suriyakr2024it/Event%20Management%20System/_apps/hub.ms.vss-build-web.ci-designer-hub?sourceProvider=github&telemetrySess...

Azure DevOps suriyakr2024it / Event Management System / Pipelines

Connect Select Configure **Review**

New pipeline

Review your pipeline YAML Variables Save and run

suria-prakash-s-it/azure-project / azure-pipelines-1.yml

```
1 # Node.js
2 # Build a general Node.js project with npm.
3 # Follow link (ctrl + click) to code, save build artifacts, deploy, and more:
4 # https://docs.microsoft.com/azure/devops/pipelines/languages/javascript
5
6 trigger:
7   - main
8
9 pool:
10  vmImage: ubuntu-latest
11
12 steps:
13   Settings
14   - task: NodeTool@0
15     inputs:
16       versionSpec: '20.x'
17       displayName: 'Install Node.js'
18
19   - script: |
20     npm install
21     npm run build
```

Azure DevOps suriyakr2024it / Event Management System / Pipelines

Search

Event Management Sy... +

- Overview
- Boards
- Repos
- Pipelines
- Pipelines
- Environments
- Library
- Test Plans
- Artifacts

Project settings <<

Pipelines

Recent All Runs

New pipeline

Filter pipelines

Recently run pipelines

Pipeline	Last run
 suriya-prakash-s-it.azure-project	#20260511.10 • fix: revert to NodeTool@0 - NodeTool@1 not available, causing wrong node v... Manually run by  main 3h ago 1m 50s

Result:

Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.

EXP NO: 10

GITHUB:PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the event management Batch Creator project.

GitHub Project Structure

The screenshot displays a GitHub repository page for the project 'software-construction-event-management-system' by user 'itmustbesomebody'. The repository is public and currently has 0 stars, 0 forks, and 0 watchers. The main branch is 'main', with 1 branch and 0 tags. The repository contains two files: 'azure-project-main.zip' and 'sc record.pdf', both added via upload 1 minute ago. A commit history shows 1 commit. The 'About' section indicates no description, website, or topics are provided. The 'Releases' section shows no published releases. The 'Packages' section shows no published packages. The 'Contributors' section lists 1 contributor: 'itmustbesomebody' (suriya narayanan). The 'README' section is currently empty, with a prompt to 'Add a README' to help people understand the project.

Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.